

Trabajo Práctico

Super Elizabeth Sis.



Materia: Programación 1, primer semestre 2026

Integrantes		
👤 Nombre	✉ Correo electrónico	🆔 DNI
Noelía Barrionuevo	nbarrionuevob26@gmail.com	37.038.375
Miguel Angel Luna Lobos	lunalobosmiguel@gmail.com	36.849.993

Introducción

En el siguiente trabajo práctico se nos presentó el desafío de desarrollar un videojuego cuyas características fueran similares a Mario Bros. Con la salvedad de que los roles se invierten y ahora la heroína es la princesa, llamada Elizabeth, y el rehén es Mario. Nuestra princesa tendrá como misión rescatar a Mario, que se encuentra cautivo en el castillo del malvado Rey Camin. Para llegar hasta el Rey, Elizabeth, debe enfrentarse con enemigos que le irán robando vidas si no logra vencerlos. Para el buen desarrollo de este video juego se deberá cumplir con los siguientes requisitos obligatorios :

- 1) La princesa Elizabeth debe aparecer en el medio de la pantalla
- 2) Islas flotantes por donde se desplace la princesa (algunas fijas y otras aleatorias)
- 3) Desplazamiento hacia la derecha e izquierda sin sobrepasar los límites de la pantalla.
- 4) Salto y caída. La princesa debe poder saltar hasta una cierta altura, caer en caída libre y además rebotar al tocar la parte inferior de las islas .
- 5) Enemigos voladores que aparezcan de ambos lados de la pantalla (izquierdo y derecho) con el fin de dañar a la princesa y quitarle vidas.
- 6) La princesa podrá realizar disparos para defenderse y matar a sus enemigos.

El juego presenta victoria cuando la Princesa llega al castillo, el cual se encontrará en el borde derecho de nuestra pantalla. Con la victoria lograda tendremos la finalización del juego. En caso contrario tendremos condición de derrota; esta se cumple cuando la princesa pierda todas sus vidas, las cuales serán 10.

Estructura del proyecto

El proyecto presenta una estructura base desde la cual se comenzó el desarrollo. Esta estructura incluye al paquete entorno.jar que contiene a la clase Entorno y también incluye la carpeta src dentro de la cual está el código del proyecto. Dentro de src existe una carpeta llamada juego que cuenta con una clase Juego, que es la clase principal del proyecto y que implementa la interfaz especificada en entorno.jar, InterfaceJuego.

La clase Entorno se ocupa de los gráficos. Controla el ciclo principal del Juego, llamando al método `tick()` del objeto de tipo InterfaceJuego que se le pasa como argumento al construir una instancia. Además, detecta teclas presionadas, movimientos del mouse y otros.

A continuación se realizará una breve descripción del flujo del programa para luego entrar en detalles sobre las clases implementadas, variables de instancias y la descripción de cada método.

Al ejecutarse la clase Juego se produce el siguiente flujo de forma ordenada:

- 1) Se ejecuta el método `main`.
- 2) Se crea una instancia de la clase Juego.
- 3) Dentro del constructor de Juego se crea un objeto de la clase Entorno.
- 4) El objeto entorno crea la ventana gráfica.
- 5) El objeto entorno inicia el ciclo principal del juego.
- 6) Durante el ciclo, el objeto entorno invoca al método `tick()`.
- 7) El método `tick()` continúa ejecutándose de forma repetitiva mientras la ventana del juego continúe abierta.

Clase Rectángulo

Las instancias de Rectángulo representan un rectángulo centrado en un punto (x, y) con un ancho y un alto determinados. Es la estructura geométrica base del juego: todos los elementos con colisión utilizan esta clase. Es inmutable (todos sus campos son final). Es utilizada por prácticamente todas las clases del juego como caja de colisión y como argumento en métodos de Rectangulos, Mundo, Isla, Princesa, Enemigo, Jefe, ProyectilPrincesa y Castillo.

Propiedades de instancia

- **x**: coordenada x del centro del rectángulo
- **y**: coordenada y del centro del rectángulo
- **ancho**: ancho del rectángulo
- **alto**: alto del rectángulo

Métodos de instancia

- **x()**: devuelve la coordenada x del rectángulo
- **y()**: devuelve la coordenada y del centro del rectángulo
- **ancho()**: devuelve el ancho del rectángulo
- **alto()**: devuelve el alto del rectángulo
- **bordeSuperior()**: devuelve $y - \text{alto}/2$ (borde superior)
- **bordeInferior()**: devuelve $y + \text{alto}/2$ (borde inferior)
- **bordeDerecho()**: devuelve $x + \text{ancho}/2$ (borde derecho)
- **bordeIzquierdo()**: devuelve $x - \text{ancho}/2$ (borde izquierdo)
- **dibujarRectangulo(Entorno entorno, Color color)**: Dibuja el rectángulo en pantalla con el color indicado
- **escalar(double ancho, double alto)**: devuelve un nuevo rectángulo con ancho y alto escalados
- **area()**: devuelve el área del rectángulo ($\text{ancho} * \text{alto}$)

Clase Rectángulos

Es una clase utilitaria con métodos estáticos para detección y clasificación de colisiones entre objetos Rectángulo. Es utilizada por Juego, Isla, Mundo y Princesa para determinar si dos entidades del juego se están tocando y qué acción tomar como consecuencia.

Métodos estáticos

- **enColision(Rectangulo r1, Rectangulo r2)**: Devuelve true si los dos rectángulos se superponen
- **tipoDeColision(Rectangulo r1, Rectangulo r2)**: Devuelve una cadena que indica desde qué lado colisionó el primer rectángulo con el segundo: "desde arriba", "desde abajo", "desde la derecha" o "desde la izquierda".

Clase Coordenadas

Es una clase utilitaria para transformar coordenadas del mundo a coordenadas relativas a la pantalla. La cámara del juego sigue a la princesa: el centro de la pantalla siempre corresponde a la posición de la princesa en el mundo. Esta clase se usa para facilitar esa transformación. Las instancias representan un simple par de coordenadas.

Métodos estáticos

- **transformar(double x, double y, Mundo mundo, Entorno entorno)**: calcula la posición en pantalla dado un punto del mundo, la posición de la princesa y las dimensiones del entorno.

Propiedades de instancia

- **x**: coordenada x
- **y**: coordenada y

Métodos de instancia

- **x()**: devuelve la coordenada x
- **y()**: devuelve la coordenada y

Clase GeneradorId

Las instancias de esta clase generan identificadores enteros únicos e incrementales. Cada llamada a `nuevoId()` devuelve un número distinto al anterior y consecutivo. Es utilizada por Juego para asignar IDs únicos a cada Enemigo al momento de su creación. Los IDs permiten a Mundo mantener el arreglo de enemigos ordenado para búsqueda binaria.

Propiedades de instancia

- **contador**: valor del próximo ID a ser devuelto

Métodos de instancia

- **nuevoId()**: devuelve el siguiente ID disponible e incrementa el contador

Clase Aleatorio

Clase utilitaria con métodos estáticos para generar números aleatorio. Encapsula el uso de Random. Es utilizada por Islas para posicionar islas aleatoriamente, por Enemigos para determinar la altura de aparición de enemigos, y por Juego para elegir de qué lado aparece cada enemigo.

Métodos estáticos

- **decimalRandom(double min, double max)**: Devuelve un double aleatorio en el intervalo [min, max).

→ **enteroRandom(int min, int max)**: devuelve un int aleatorio en el intervalo [min, max)

Clase Imagenes

Clase utilitaria para cargar y escalar imágenes. Depende de que los archivos `.png` estén en el mismo directorio que los `class` compilados.

Métodos estáticos

- **cargar(String nombreArchivo)**: Carga una imagen desde el classpath relativo a la clase `Imagenes`
- **cargarYEscalar(String nombreArchivo, double w, double h)**: Carga una imagen y la escala a las dimensiones indicadas
- **escalar(Image imagen, double ancho, double alto)**: Escala una imagen ya cargada a las dimensiones indicadas

Clase IteradorIslas

Implementa un iterador simple sobre un arreglo de islas. Hace una copia interna del arreglo al construirse.

Propiedades de instancia

- **islas**: copia interna del arreglo de islas
- **indice**: posición actual del iterador
- **largo**: cantidad de islas válidas en el arreglo

Métodos de instancia

- **tieneOtro()**: devuelve true si quedan islas por recorrer
- **proximo()**: devuelve la siguiente isla y avanza en el índice

Clase IteradorEnemigos

Implementa un iterador simple sobre un arreglo de enemigos. Al igual que IteradorIslas, copia el arreglo al construirse.

Propiedades de instancia

- **enemigos**: copia interna del arreglo de enemigos
- **indice**: posición actual del iterador
- **largo**: cantidad de enemigos válidos en el arreglo

Métodos de instancia

- **tieneOtro()**: devuelve true si quedan enemigos por recorrer
- **proximo()**: devuelve el siguiente enemigo y avanza el índice

Clase Fondo

Representa el fondo visual del juego. Es una imagen estática que se dibuja siempre en la misma posición de pantalla, independientemente del movimiento de la cámara.

Propiedades de instancia

- **x**: coordenada x
- **y**: coordenada y
- **fondo**: imagen de fondo

Métodos de instancia

- **dibujar(Entorno entorno)**: dibuja el fondo en su posición fija

Clase Isla

La clase Isla modela una plataforma flotante del escenario. Almacena su posición, tamaño e image. Además se encarga de dibujar y de informar a otros objetos del juego cuando ocurre una colisión con ella, detectando de qué lado se produce el choque y envía un mensaje al objeto involucrado, para que éste pueda reaccionar de forma adecuada. Por ejemplo informa a la princesa cuando se encuentra sobre tierra firme o al proyectil cuando debe rebotar. Para facilitar estas detecciones, la isla puede generar un Rectángulo que representa el área que ocupa dentro del mundo. Este rectángulo es utilizado por el sistema de colisión para determinar las interacciones con otras identidades.

Propiedades de instancia

- **x**: coordenada x del centro de la isla
- **y**: coordenada y del centro de la isla
- **ancho**: ancho de la isla
- **alto**: alto de la isla
- **isla**: imagen de la isla

Métodos de instancia

- **x()**: devuelve la coordenada x del centro de la isla
- **y()**: devuelve la coordenada y del centro de la isla
- **dibujar(Entorno entorno, Mundo mundo)**: dibuja la isla transformando sus coordenadas
- **actuarSobrePrincesa(Princesa princesa)**: detecta el tipo de colisión y envía el mensaje correspondiente a la princesa.
- **actuarSobreProyectilPrincesa(ProyectilPrincesa proyectilPrincesa)**: detecta el tipo de colisión y ordena al proyectil rebotar en la dirección correcta.

Clase Islas

Clase de fábrica con métodos estáticos, se encarga de generar islas válidas y distribuirlas dentro del mundo. Define dos tipos: islas de nivel bajo (en la mitad inferior del mapa, más angostas y ancho fijo) e islas de nivel alto (en la mitad superior, más anchas pero de anchos aleatorios). Garantiza que ninguna isla nueva colisione con las ya existentes probando hasta 1000 veces. Es utilizada por Juego durante la inicialización del mundo para poblar el mapa con plataformas.

Propiedades estáticas

- **altoIsla**: altura de todas las islas
- **anchoMínimo**: ancho mínimo de las islas de nivel alto
- **anchoMaximo**: ancho máximo de las islas de nivel alto
- **niveles**: número de niveles de islas
- **proporcionNivelesAltos**: fracción de niveles considerados altos
- **factorFronteraAncho**: factor de expansión horizontal para detectar solapamiento con otras islas
- **proporcionIslasBajas**: densidad de islas bajas respecto al área del mundo
- **proporcionIslasAltas**: densidad de islas altas respecto al área del mundo

Métodos estáticos

- **nuevaNivelBajo(Mundo mundo)**: genera una isla de nivel bajo sin solapamientos
- **nuevaNivelAlto(Mundo mundo)**: genera una isla de nivel alto sin solapamientos
- **islaAleatoriaNivelBajo(Mundo mundo)**: genera una isla de nivel bajo sin verificar solapamiento
- **islaAleatoriaNivelAlto(Mundo mundo)**: genera una isla de nivel alto sin verificar solapamiento

Clase Castillo

Representa el castillo al final del nivel. La princesa debe alcanzarlo para ganar. Es el objeto objetivo del juego. Su rectángulo de colisión usa solo el 20% del ancho total para requerir que la princesa esté bien posicionada al tocarlo.

Propiedades de instancia

- **x**: coordenada x del centro del castillo
- **y**: coordenada y del centro del castillo
- **ancho**: ancho del castillo
- **alto**: alto del castillo
- **castillo**: la imagen del castillo

Métodos de instancia

- **dibujar(Entorno entorno, Mundo mundo)**: dibuja el castillo aplicando una transformación de coordenadas

- **trasladar(double x, double y)**: cambia la posición del castillo
- **alto()**: devuelve el alto del castillo
- **rectángulo()**: devuelve el rectángulo de colisión del castillo

Clase Enemigo

Representa a un enemigo ordinario del juego. Se mueve horizontalmente con velocidad constante. Tiene un identificador único para poder ser encontrado eficientemente en el arreglo ordenado de Mundo. Al recibir el mensaje "morir" se marca para ser eliminado. Las instancias se crean en Enemigos, se almacenan en Mundo en un arreglo ordenado por ID. Juego lo dibuja, mueve y gestiona sus colisiones con la princesa y los proyectiles.

Propiedades de instancia

- **id**: entero que hace las veces de identificador único
- **x**: coordenada x del centro del enemigo
- **y**: coordenada y del centro del enemigo
- **ancho**: ancho del enemigo
- **alto**: alto del enemigo
- **ángulo**: ángulo de movimiento
- **velocidad**: velocidad de desplazamiento
- **vivo**: booleano que indica si el enemigo sigue vivo
- **imagen**: imagen del enemigo

Métodos de instancia

- **id()**: devuelve el identificador único del enemigo
- **dibujar(Entorno entorno, Mundo mundo)**: dibuja al enemigo transformado las coordenadas
- **mover()**: desplaza al enemigo según su ángulo y velocidad
- **recibirMensaje(String mensaje)**: acepta "morir" para marcar al enemigo como muerto
- **debeEliminarse()**: devuelve true si el enemigo está muerto
- **rectángulo()**: devuelve el rectángulo de colisión

Clase Enemigos

Clase de fábrica con métodos estáticos para crear enemigos en los bordes de la pantalla. Los enemigos aparecen a una altura aleatoria correspondiente a uno de los niveles de isla y avanzan hacia el lado opuesto.

Propiedades estáticas

- **altoEnemigo**: alto de todos enemigos
- **anchoEnemigo**: ancho de todos los enemigos
- **minimoEnemigos**: mínimo de enemigos activos en pantalla

Métodos estáticos

- **nuevoEnemigoDerecha(GeneradorId generadorId, Mundo mundo, Entorno entorno)**: crea un enemigo que entra por el borde derecho y avanza a la izquierda
- **nuevoEnemigoIzquierda(GeneradorId generadorId, Mundo mundo, Entorno entorno)**: crea un enemigo que entra por el borde izquierdo y avanza a la derecha
- **enemigoAleatorio(GeneradorId, Mundo, double x, double angulo)**: crea un enemigo en la posición dada con la dirección indicada
- **alturaAleatoria(int nivel, double altoMundo, double yMin)**: calcula la coordenada y correspondiente a un nivel de isla dado

Clase ProyectilPrincesa

Representa el proyectil que dispara la princesa al hacer clic izquierdo con el mouse. Se mueve en línea recta hacia el cursor al momento del disparo. Rebota en las islas invirtiendo la componente de velocidad correspondiente al eje de la colisión.

Propiedades de instancia

- **x**: coordenada x del centro del proyectil
- **y**: coordenada y del centro del proyectil
- **ancho**: ancho del proyectil
- **alto**: alto del proyectil
- **cos**: componente horizontal del vector de dirección unitario
- **sen**: componente vertical del vector de dirección unitario
- **velocidad**: módulo de la velocidad del proyectil
- **imagen**: imagen del proyectil

Métodos de instancia

- **dibujar(Entorno entorno, Mundo mundo)**: dibuja el proyectil aplicando la transformación de coordenadas
- **mover()**: actualiza la posición del proyectil sumando $velocidad * cos$ y $velocidad * sen$ a x e y respectivamente
- **recibirMensaje(String mensaje)**: acepta "rebotar" lo cual invierte sen o cos según la dirección de colisión
- **rectangulo()**: devuelve el rectángulo de colisión del proyectil

Clase Princesa

Es el personaje principal del juego. Se mueve horizontalmente con las teclas de dirección, salta con la tecla A o flecha arriba, y dispara proyectiles hacia el cursor al hacer clic izquierdo. Implementa simulación de gravedad y caída libre con marcas temporales. Tiene un sistema de vidas representado por corazones en pantalla.

Propiedades estáticas

- **altoPrincesa**: alto configurado de la princesa
- **anchoPrincesa**: ancho configurado de la princesa
- **ladoCorazon**: el lado del cuadrado de cada corazón

Propiedades de instancia

- **x**: la coordenada x del centro de la princesa
- **y**: la coordenada y del centro de la princesa
- **ancho**: ancho de la princesa
- **alto**: alto de la princesa
- **velocidad**: velocidad de movimiento lateral
- **velocidadSalto**: velocidad vertical inicial al saltar
- **aceleracionGravitatoria**: aceleración de gravedad
- **velocidadCaídaLibre**: velocidad vertical acumulada en caída
- **enSalto**: indica si la princesa está ejecutando un salto
- **xNoCrece**: booleano que indica si la princesa se puede mover hacia la derecha
- **xNoDecrece**: booleano que indica si la princesa se puede mover hacia la izquierda
- **vidas**: vidas restantes
- **princesa**: imagen de la princesa
- **corazon**: imagen de un corazón usado para ilustrar las vidas de la princesa

Métodos de instancia

- **x()**: devuelve la coordenada x del centro de la princesa
- **y()**: devuelve la coordenada y del centro de la princesa
- **ancho()**: devuelve el ancho de la princesa
- **alto()**: devuelve el alto de la princesa
- **vidas()**: devuelve las vidas restantes de la princesa
- **trasladar(double x, double y)**: teletransporta a la princesa hacia las coordenadas del punto (x,y)
- **dibujar(Entorno entorno)**: dibuja a la princesa en el centro de la pantalla y dibuja los corazones que indican la cantidad de vidas restantes en la esquina superior izquierda de la pantalla
- **mover(Entorno entorno)**: aplica el movimiento lateral, salto y gravedad según el estado de las teclas y de la princesa
- **disparar(Mundo mundo, Entorno entorno)**: calcula el vector hacia el cursor y crea un `ProyectilPrincesa`
- **recibirMensaje(String mensaje)**: procesa mensajes de colisión y daño
- **rectangulo()**: devuelve el rectángulo de colisión de la princesa

Clase Jefe

Representa al enemigo jefe del nivel. Patrulla de lado a lado sobre una isla específica, cambiando de dirección al llegar a los bordes. Tiene 10 vidas y una barra de salud visible en pantalla. Matar al

jefe (disparándole con el proyectil) no es condición de victoria, pero es el obstáculo más peligroso: tocarlo mata a la princesa instantáneamente.

Propiedades estáticas

- **altoJefe**: el alto configurado del jefe
- **anchoJefe**: el ancho configurado del jefe

Propiedades de instancia

- **x**: la coordenada x del centro del jefe
- **y**: la coordenada y del centro del jefe
- **ancho**: el ancho del jefe
- **alto**: el alto del jefe
- **cos**: componente horizontal del vector de dirección
- **velocidad**: el módulo de la velocidad
- **vidas**: las vidas restantes del jefe
- **isla**: isla sobre la cual patrulla el jefe
- **jefeHaciaDerecha**: imagen del jefe cuando avanza hacia la derecha
- **jefeHaciaIzquierda**: imagen del jefe cuando avanza hacia la izquierda
- **jefe**: imagen actual del jefe

Métodos de instancia

- **establecerIsla(Isla isla)**: coloca al jefe sobre la isla indicada
- **x()**: devuelve la coordenada x del jefe
- **y()**: devuelve la coordenada y del jefe
- **ancho()**: devuelve el ancho del jefe
- **alto()**: devuelve el alto del jefe
- **vidas()**: devuelve las vidas restantes del jefe
- **dibujar(Entorno entorno, Mundo mundo)**: dibuja al jefe con la barra de salud superpuesta
- **mover()**: desplaza al jefe e invierte su dirección cuando llega al borde de la isla en la que patrulla
- **recibirMensaje(String mensaje)**: acepta “una vida menos” para reducir su cantidad de vidas
- **rectangulo()**: devuelve el rectángulo de colisión

Clase Mundo

Contenedor central del estado del juego. Almacena todas las entidades activas (princesa, jefe, enemigos, islas, proyectil, castillo, fondo) y provee operaciones de consulta y modificación. Gestiona el arreglo dinámico de enemigos (con redimensionamiento automático) y el arreglo de islas. Los enemigos se mantienen ordenados por ID para permitir búsqueda binaria en

`quitarEnemigo()`. Se trata del objeto central al que acceden ``Juego``, ``Isla``, ``Islas``, ``Enemigos``, ``Coordenadas`` y ``Princesa``. Actúa como repositorio compartido de estado de juego.

Propiedades estáticas

- **proporcionAnchoMundo**: la cantidad de veces que entra la pantalla a lo ancho del mundo
- **proporcionAltoMundo**: la cantidad de veces que entra la pantalla a lo alto del mundo

Propiedades de instancia

- **princesa**: el personaje jugable
- **jefe**: el jefe del nivel, puede ser null si fue eliminado
- **castillo**: el castillo objetivo
- **fondo**: el fondo del escenario
- **proyectilPrincesa**: el proyectil disparado por la princesa, puede ser null
- **enemigos**: un arreglo de enemigos ordenados de manera natural por id que se dibujan en pantalla y no se solapan entre sí al crearse
- **islas**: un arreglo de islas que se dibujan en pantalla y que no se solapan entre sí
- **largoEnemigos**: la cantidad de enemigos que se albergan en el array enemigos
- **largoIslas**: la cantidad de islas que se albergan en el array islas
- **limitesMundo**: un rectángulo que representa al mundo completo
- **limitesPantalla**: un rectángulo que representa la pantalla inicial

Métodos de instancia

- **limitesMundo()**: devuelve el rectángulo con los límites del mundo
- **limitesPantalla()**: devuelve el rectángulo con los límites actuales de la pantalla en las coordenadas del mundo
- **fondo()**: devuelve el objeto fondo
- **princesa()**: devuelve el objeto princesa
- **castillo()**: devuelve el objeto castillo
- **jefe()**: devuelve el objeto jefe o null si no existe
- **proyectilPrincesa()**: devuelve el objeto proyectilPrincesa o null si no existe
- **agregarEnemigo(Enemigo enemigo)**: agrega un enemigo y redimensiona enemigos si es necesario.
- **agregarIsla(Isla isla)**: agrega una isla y redimensiona islas si es necesario.
- **quitarEnemigo(Enemigo enemigo)**: busca al enemigo con búsqueda binaria y lo elimina del arreglo
- **enemigosEnColision(Rectangulo rectangulo)**: devuelve los enemigos que colisionan con el rectángulo provisto
- **islasEnColision(Rectangulo rectangulo)**: devuelve las islas en colisión con el rectángulo provisto
- **iteradorIslas()**: devuelve un objeto IteradorIslas que itera sobre islas
- **iteradorEnemigos()**: devuelve un objeto IteradorEnemigos que itera sobre enemigos
- **purgar()**: elimina enemigos fuera de pantalla o muertos, también al jefe y al proyectil de la princesa si corresponde

- **establecerProyectilPrincesa(ProyectilPrincesa proyectilPrincesa):**
establece o elimina el proyectil
- **faltanEnemigos():** devuelve true si hay menos enemigos activos que `minimoEnemigos`

Clase Juego

Es la clase principal del juego. Implementa el contrato de `InterfaceJuego` provisto por la biblioteca `entorno.jar`. Inicializa todos los objetos del mundo en el constructor y ejecuta el bucle de juego frame a frame en el método `tick()`. Coordina rendering, detección de colisiones, creación de enemigos y condiciones de victoria/derrota.

Propiedades de instancia

- **entorno:** motor gráfico y de entrada de la biblioteca externa
- **generadorId:** generador de IDs únicos para enemigos
- **mundo:** estado del juego
- **victoria:** indica si la princesa llegó al castillo
- **derrota:** indica si la princesa quedó sin vidas

Métodos de instancia

tick(): especificado por `InterfaceJuego` dentro de éste método se ejecuta la lógica continua del juego. Se dibuja, se mueven elementos, se detectan colisiones, se generan enemigos y se verifican condiciones de fin de juego.

dibujarEnemigos(): mueve y dibuja cada enemigo activo

colisionesIslas(): gestiona colisiones de islas con la princesa y el proyectil, y dibuja las islas

colisionesJefe(): detecta colisión del jefe con la princesa (muerte instantánea) y dibuja al jefe

colisionesProyectilPrincesa(): gestiona colisiones del proyectil con el jefe y los enemigos

colisionesPrincesa(): detecta colisiones de la princesa con enemigos, comprueba límites y la mueve

colisionesCastillo(): verifica si la princesa tocó el castillo (victoria) y dibuja el castillo

princesaCaeAlVacio(): resta una vida y reaparece a la princesa en la isla más cercana

main(String[] args): punto de entrada del programa, instancia `Juego` que en su constructor llama al método `iniciar()` de `entorno`

Flujo general del método tick()

- Dibujar fondo
- Si hay victoria o derrota mostrar mensaje, congelar princesa, retornar
- Crear enemigos si faltan
- Purgar entidades fuera de pantalla o muertas
- Procesar disparo de la princesa (clic izquierdo)
- Llamar a `colisionesIslas()`
- Llamar a `colisionesJefe()`

- Llamar a `colisionesProyectilPrincesa()`
- Llamar a `dibujarEnemigos()`
- Llamar a `colisionesCastillo()`
- Llamar a `colisionesPrincesa()`

Conflictos durante el desarrollo del juego

Creación de enemigos

Durante el desarrollo de la clase `Enemigos`, surgió una dificultad importante. En una primera versión del proyecto los enemigos fueron creados individualmente mediante variables independientes, sin almacenarse en una estructura común. Esta decisión complicaba la detección de colisiones, la eliminación de enemigos y el control de la cantidad mínima de enemigos presentes en pantalla, ya que cada operación debía realizarse sobre cada objeto de forma separada.

Para solucionar este problema se decidió almacenar los enemigos dentro de un arreglo, administrado por la clase `Mundo`. Gracias a esa modificación fue posible recorrer todos los enemigos mediante iteradores, verificar colisiones de forma uniforme y agregar o eliminar enemigos dinámicamente durante la ejecución del juego sin tener que manejar variables individuales.

El siguiente método fue posible pensarlo porque pasamos de tener enemigos individuales a tener una colección de enemigos administrados por `Mundo`. Lo que hace este método es verificar si el arreglo está lleno y si es así, crea un arreglo más grande, copia los enemigos del arreglo viejo al nuevo y reemplaza el array viejo.

```
public void agregarEnemigo(Enemigo enemigo) {
    if (largoEnemigos == enemigos.length) {
        Enemigo[] nuevoArray = new Enemigo[enemigos.length * 2];
        System.arraycopy(enemigos, 0, nuevoArray, 0, enemigos.length);
        enemigos = nuevoArray;
    }
    enemigos[largoEnemigos++] = enemigo;
}
```

Además, la forma de generar enemigos es tal que sigue la siguiente formulación matemática, de manera que los enemigos aparezca en lugares donde no colisionen con las islas al desplazarse de manera vertical.

Sea n la cantidad de niveles de islas, e “ y ” las altura posibles los enemigos se generaran en alturas que siguen esta expresión

$$y = (2 * q + 1) * (y_{max} - y_{min}) / (2 * (n - 1)) + y_{min}$$

Donde:

$$q \in [0, ..., n - 2)$$

y_{min} : es la altura mínima

y_{max} : es la altura máxima

Creación de islas flotantes

Otro conflicto presentado fue en la creación de las islas flotantes. Cada isla era creada como un objeto independiente (isla1, isla2, isla3 ...), algo muy similar a la creación de enemigos. La lógica para evitar que se superpusieran debía programarse manualmente para cada nueva isla. Por ejemplo la segunda isla debía verificar su posición respecto de la primera, la tercera debía compararse con la primera y la segunda y así sucesivamente. A medida que aumentaba la cantidad de islas, el código se volvía extenso y difícil de mantener, ya que cada nueva isla requería agregar nuevas comparaciones.

Como solución se decidió almacenar las islas dentro de un arreglo, administrado por la clase Mundo y al igual que en la creación de enemigos, fue posible recorrer todas las islas mediante iteradores. De esa forma se facilitó verificar colisiones de forma general, sin necesidad de escribir condiciones específicas para cada objeto. La lógica de creación aleatoria continuó respetando la restricción de que las islas no pueden superponerse, pero ahora dicha validación se realiza recorriendo el conjunto de islas existentes.

Esta solución simplificó considerablemente el código, facilitó la incorporación de nuevas islas y permitió reutilizar los mismos mecanismos de detección de colisiones utilizados por otros elementos del juego. Además, se decidió generar las islas a alturas que siguen una fórmula matemática. Sean “y” las alturas posibles de las islas y “n” la cantidad de niveles de islas, estas siguen la siguiente expresión

$$y = q * (y_{max} - y_{min}) / (n - 1) + y_{min}$$

Donde

$$q \in [0, n)$$

y_{min} : es la altura mínima

y_{max} : es la altura máxima

Reproducción de sonido

Durante la colocación del sonido se presentó un inconveniente relacionado con la reproducción de efectos de sonido. Inicialmente, algunos sonidos fueron ubicados dentro de condiciones que se evaluaban en cada ejecución del método tick(). Dado que el ciclo principal del juego se ejecuta aproximadamente cada 15 a 30 milisegundos, la condición permanecía verdadera durante varios ciclos consecutivos. Como consecuencia, el mismo archivo de audio intentaba reproducir múltiples veces en un intervalo muy corto de tiempo. Este comportamiento provocó que varias instancias del sonido se superpusieron, generando una reproducción distorsionada, poco clara.

La solución consistió en reproducir los sonidos únicamente cuando ocurría el evento que los generaba; ejemplo, una colisión o la pérdida de vidas; y no durante cada evaluación de la condición. De esta manera, cada efecto de sonido se ejecuta una sola vez por evento, evitando superposiciones y mejorando la calidad de la experiencia auditiva.

Detección de tipo de colisión

Otro problema fue detectar desde que parte se colisiona un elemento con otro. Se llegó a la conclusión, tal vez no del todo muy eficiente, de que se deben probar primero las 4 condiciones, ya que siempre coexisten al menos 2 de ellas. De ahí se debe determinar cual es la que domina la

naturaleza de la colisión. Eso se logró determinando los segmentos de las aristas en colisión y viendo cual es el mas largo, si es vertical se trata de una colisión de naturaleza horizontal, si es el de horizontal se trata de una colisión de naturaleza vertical. Se implementó esta lógica de la siguiente manera

```
public static String tipoDeColision(Rectangulo r1, Rectangulo r2) {
    final double x1 = r1.x();
    final double x2 = r2.x();
    final double y1 = r1.y();
    final double y2 = r2.y();
    final double bordeIzquierdo1 = r1.bordeIzquierdo();
    final double bordeDerecho1 = r1.bordeDerecho();
    final double bordeSuperior1 = r1.bordeSuperior();
    final double bordeInferior1 = r1.bordeInferior();
    final double bordeIzquierdo2 = r2.bordeIzquierdo();
    final double bordeDerecho2 = r2.bordeDerecho();
    final double bordeSuperior2 = r2.bordeSuperior();
    final double bordeInferior2 = r2.bordeInferior();

    double deltaY = 0.0;
    double deltaX = 0.0;
    boolean desdeArriba = bordeInferior1 >= bordeSuperior2 && y1 <= y2;
    boolean desdeAbajo = bordeSuperior1 <= bordeInferior2 && y1 >= y2;
    boolean desdeLaDerecha = bordeDerecho1 >= bordeIzquierdo2 && x1 <= x2;
    boolean desdeLaIzquierda = bordeIzquierdo1 <= bordeDerecho2 && x1 >= x2;

    if (desdeArriba) {
        deltaY = Math.min(bordeInferior1 - bordeSuperior2, r1.alto());
    }

    if (desdeAbajo) {
        deltaY = Math.min(bordeInferior2 - bordeSuperior1, r1.alto());
    }

    if (desdeLaDerecha) {
        deltaX = Math.min(bordeDerecho1 - bordeIzquierdo2, r1.ancho());
    }

    if (desdeLaIzquierda) {
        deltaX = Math.min(bordeDerecho2 - bordeIzquierdo1, r1.ancho());
    }

    if (deltaY >= deltaX) { // lateral
        if (desdeLaDerecha) {
            return "desde la derecha";
        } else if (desdeLaIzquierda) {
            return "desde la izquierda";
        }
    } else { // vertical
        if (desdeArriba) {
            return "desde arriba";
        } else if (desdeAbajo) {
            return "desde abajo";
        }
    }
}
```



```
        }  
    }  
  
    throw new IllegalArgumentException("no hay colisión");  
}
```

Conclusión

Durante el desarrollo del proyecto, se implementaron los conocimientos adquiridos durante la cursada; la creación de objetos, métodos y la organización de la lógica en múltiples clases con responsabilidades específicas. Además, nos permitió comprender cómo distintos objetos pueden colaborar entre sí para construir un sistema más complejo.

La clase Juego actúa como punto de inicio y coordina la ejecución general mediante el método tick (), mientras que la clase Mundo administra los distintos elementos que forman parte de la partida, controla su almacenamiento, eliminación y las consultas necesarias para detectar colisiones.

Las entidades principales del juego, como princesa , jefe, isla , enemigo poseen su propio comportamiento, movimiento y representación gráfica. Asimismo, la utilización de las clases auxiliares permiten centralizar tareas específicas, favoreciendo a una mejor organización del programa.

La utilización de rectángulos para representar áreas de colisión permitió implementar un sistema uniforme de interacción entre los distintos objetos del juego. Del mismo modo , el empleo de iteradores, generadores de identificadores y clases de creación contribuyó a mantener una estructura ordenada.

Como resultado se obtuvo un juego funcional en el que la protagonista puede desplazarse, saltar, interactuar con plataformas, enfrentarse a enemigos y utilizar proyectiles gracias a la colaboración entre las distintas clases que componen el sistema.

Implementación

Clase Rectángulo

```
package juego;
import entorno.Entorno;
import java.awt.*;
/**
 * Clase para trabajar con colisiones.
 *
 * @author Miguel Angel Luna Lobos
 */
public class Rectangulo {
    private final double x;
    private final double y;
    private final double ancho;
    private final double alto;
    public Rectangulo(double x, double y, double ancho, double alto) {
        super();
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
    }
    public double x() { return x; }
    public double y() { return y; }
    public double ancho() { return ancho; }
    public double alto() { return alto; }
    /**
     * @return la coordenada y del borde superior (y - alto / 2)
     */
    double bordeSuperior() { return y() - alto() / 2.0; }
    /**
     * @return la coordenada y del borde inferior (y + alto / 2)
     */
    double bordeInferior() { return y() + alto() / 2.0; }
    /**
     * @return la coordenada x del borde derecho (x + ancho / 2)
     */
    double bordeDerecho() { return x() + ancho() / 2.0; }
    /**
     * @return la coordenada x del borde izquierdo (x - ancho / 2)
     */
    double bordeIzquierdo() { return x() - ancho() / 2.0; }
    /**
     * Dibuja el rectángulo con un color dado.
     *
     * @param entorno el entorno
     * @param color el color de relleno
     */
    void dibujarRectangulo(Entorno entorno, Color color) {
```

```

        entorno.dibujarRectangulo(x(), y(), ancho(), alto(), 0.0, color);
    }
    public Rectangulo escalar(double proporcionX, double proporcionY) {
        return new Rectangulo(x, y, ancho * proporcionX, alto * proporcionY);
    }
    /**
     * Devuelve el área del rectángulo.
     * @return el área del rectángulo
     */
    public double area(){
        return ancho() * alto();
    }
}
}

```

Clase Rectángulos

```

package juego;
/**
 * Clase para métodos utilitarios de rectángulos.
 *
 * @author Miguel Angel Luna Lobos
 */
public class Rectangulos {
    /**
     * Determina la dirección desde la que r1 llega a colisionar con r2.
     *
     * @param r1 el rectángulo cuya dirección de llegada se determina
     * @param r2 el rectángulo impactado
     * @return "desde arriba", "desde abajo", "desde la izquierda"
     * "desde la derecha"
     * @throws IllegalArgumentException si los rectángulos no están en colisión
     */
    public static String tipoDeColision(Rectangulo r1, Rectangulo r2) {
        final double x1 = r1.x();
        final double x2 = r2.x();
        final double y1 = r1.y();
        final double y2 = r2.y();
        final double bordeIzquierdo1 = r1.bordeIzquierdo();
        final double bordeDerecho1 = r1.bordeDerecho();
        final double bordeSuperior1 = r1.bordeSuperior();
        final double bordeInferior1 = r1.bordeInferior();
        final double bordeIzquierdo2 = r2.bordeIzquierdo();
        final double bordeDerecho2 = r2.bordeDerecho();
        final double bordeSuperior2 = r2.bordeSuperior();
        final double bordeInferior2 = r2.bordeInferior();
        double deltaY = 0.0;
        double deltaX = 0.0;
        boolean desdeArriba = bordeInferior1 >= bordeSuperior2 && y1 <= y2;
        boolean desdeAbajo = bordeSuperior1 <= bordeInferior2 && y1 >= y2;
        boolean desdeLaDerecha = bordeDerecho1 >= bordeIzquierdo2 && x1 <= x2;
        boolean desdeLaIzquierda = bordeIzquierdo1 <= bordeDerecho2 && x1 >= x2;
        if (desdeArriba) {

```

```

        deltaY = Math.min(bordeInferior1 - bordeSuperior2, r1.alto());
    }
    if (desdeAbajo) {
        deltaY = Math.min(bordeInferior2 - bordeSuperior1, r1.alto());
    }
    if (desdeLaDerecha) {
        deltaX = Math.min(bordeDerecho1 - bordeIzquierdo2, r1.ancho());
    }
    if (desdeLaIzquierda) {
        deltaX = Math.min(bordeDerecho2 - bordeIzquierdo1, r1.ancho());
    }
    if (deltaY >= deltaX) { // lateral
        if (desdeLaDerecha) {
            return "desde la derecha";
        } else if (desdeLaIzquierda) {
            return "desde la izquierda";
        }
    }
    else { // vertical
        if (desdeArriba) {
            return "desde arriba";
        } else if (desdeAbajo) {
            return "desde abajo";
        }
    }
    }
    throw new IllegalArgumentException("no hay colisión");
}
/**
 * Detecta si dos rectángulos se encuentran o no en colisión.
 * @param r1 el primer rectángulo
 * @param r2 el segundo rectángulo
 * @return true si están en colisión o false de lo contrario
 */
public static boolean enColision(Rectangulo r1, Rectangulo r2) {
    final double x1 = r1.x();
    final double x2 = r2.x();
    final double y1 = r1.y();
    final double y2 = r2.y();
    final double bordeIzquierdo1 = r1.bordeIzquierdo();
    final double bordeDerecho1 = r1.bordeDerecho();
    final double bordeSuperior1 = r1.bordeSuperior();
    final double bordeInferior1 = r1.bordeInferior();
    final double bordeIzquierdo2 = r2.bordeIzquierdo();
    final double bordeDerecho2 = r2.bordeDerecho();
    final double bordeSuperior2 = r2.bordeSuperior();
    final double bordeInferior2 = r2.bordeInferior();
    return ((bordeDerecho1 >= bordeIzquierdo2 && x1 <= x2) || (bordeIzquierdo1 <=
bordeDerecho2 && x1 >= x2))
        && ((bordeSuperior1 <= bordeInferior2 && y1 >= y2) || (bordeInferior1 >=
bordeSuperior2 && y1 <= y2));
}
}

```

Clase coordenadas

```
package juego;
import entorno.Entorno;
/**
 * Clase usada para guardar coordenadas x, y. Pensada para trabajar
 * con transformaciones entre sistemas de coordenadas.
 *
 * @author Miguel Angel Luna Lobos
 */
public class Coordenadas {
    /**
     * Transforma el punto x, y provisto por uno relativo a la ubicación de la princesa.
     * @param x la coordenada x
     * @param y la coordenada y
     * @param mundo el mundo
     * @param entorno el entorno
     * @return las coordenadas relativas a la princesa
     */
    public static Coordenadas transformar(double x, double y, Mundo mundo, Entorno
entorno) {
        final var rectanguloPrincesa = mundo.princesa().rectangulo();
        final var dx = entorno.anch() / 2.0;
        final var dy = entorno.alto() / 2.0;
        return new Coordenadas(x - rectanguloPrincesa.x() + dx, y -
rectanguloPrincesa.y() + dy);
    }
    private final double x;
    private final double y;
    public Coordenadas(double x, double y) {
        this.x = x;
        this.y = y;
    }
    /**
     * La coordenada x.
     * @return la coordenada x
     */
    public double x() { return x; }
    /**
     * La coordenada y.
     * @return la coordenada y
     */
    public double y() { return y; }
}
```

Clase GeneradorId

```
package juego;
/**
 * Generador de identificadores únicos e incrementales para los elementos del juego.
 * Cada llamada a nuevoId() devuelve un valor mayor al anterior, comenzando en cero.
 *
 * @author Miguel Angel Luna Lobos
 */
```

```

*/
public class GeneradorId {
    private int actual;
    /**
     * Crea un nuevo generador cuyo primer identificador será cero.
     */
    public GeneradorId(){ actual = 0; }
    /**
     * Genera y retorna el próximo identificador disponible.
     *
     * @return un entero único no negativo
     */
    public int nuevoId(){ return actual++; }
}

```

Clase Aleatorio

```

package juego;
import java.util.Random;
/**
 * Clase con métodos utilitarios para número aleatorios.
 *
 * @author Miguel Angel Luna Lobos
 */
public class Aleatorio {
    private static final Random random = new Random();
    /**
     * Devuelve un número decimal aleatorio entre los límites especificados.
     * @param min el mínimo
     * @param max el máximo
     * @return el número de aleatorio
     */
    public static double decimalRandom(double min, double max){
        if(min > max){ throw new IllegalArgumentException("max debe ser mayor a min"); }
        final var rango = max - min;
        return random.nextDouble() * rango + min;
    }
    /**
     * Devuelve un entero aleatorio que va desde min inclusive hasta max (no inclusive).
     *
     * @param min el valor mínimo (incluido)
     * @param max el máximo (excluido)
     * @return el entero aleatorio
     */
    public static int enteroRandom(int min, int max){
        if(min > max){ throw new IllegalArgumentException("max debe ser mayor a min"); }
        final var rango = max - min;
        // max - min - 1 + min = max - 1
        if(rango == 0) { return min; }
        return random.nextInt(rango) + min;
    }
}
}

```

Clase Imagenes

```
package juego;
import javax.swing.*;
import java.awt.*;
import java.util.Objects;
/**
 * Clase con métodos utilitarios de imágenes.
 *
 * @author Miguel Angel Luna Lobos
 */
public class Imagenes {
    /**
     * Carga una imagen y le da las dimensiones especificadas.
     * @param nombreArchivo el nombre del archivo
     * @param ancho el ancho
     * @param alto el alto
     * @return un objeto Image con la imagen provista y las dimensiones indicadas
     */
    public static Image cargarYEscalar(String nombreArchivo, double ancho, double alto){
        return escalar(cargar(nombreArchivo), ancho, alto);
    }
    /**
     * Método para cargar imágenes que se encuentran en la carpeta src/juego (al mismo
     nivel que las clases).
     * Lo creamos debido a que los utilitarios provistos no nos funcionaban.
     * @param nombreArchivo el nombre del archivo de la imagen
     * @return un objeto Image con la imagen provista
     * @throws NullPointerException si el archivo no existe
     */
    public static Image cargar(String nombreArchivo){
        var url = (Imagenes.class).getResource(nombreArchivo);
        Objects.requireNonNull(url, "el %s archivo no existe".formatted(nombreArchivo));
        return new ImageIcon(url).getImage();
    }

    /**
     * Escala una imagen al ancho y alto provistos.
     * @param imagen la imagen a escalar
     * @param ancho el nuevo ancho de la imagen
     * @param alto el nuevo alto de la imagen
     * @return la imagen escalada
     */
    public static Image escalar(Image imagen, double ancho, double alto){
        return imagen.getScaledInstance((int) ancho, (int) alto, Image.SCALE_DEFAULT);
    }
}
```

Clase IteradorIslas

```
package juego;
/**
 * Clase creada para iterar sobre un array de islas.
```

```

*
* @author Miguel Angel Luna Lobos
*/
public class IteradorIslas {
    private final Isla[] islas;
    private final int largo;
    private int indice;
    /**
     * Crea un iterador de islas.
     * @param islas el array de islas a recorrer
     * @param largo la cantidad de islas dentro del array a recorrer
     */
    public IteradorIslas(Isla[] islas, int largo) {
        this.islas = new Isla[largo];
        System.arraycopy(islas, 0, this.islas, 0, largo);
        this.largo = largo;
        indice = 0;
    }
    public boolean tieneOtro() { return indice < largo; }
    public Isla proximo(){ return islas[indice++]; }
}

```

Clase IteradorEnemigos

```

package juego;
/**
 * Clase creada para iterar sobre un array de enemigos.
 *
 * @author Miguel Angel Luna Lobos
 */
public class IteradorEnemigos {
    private final Enemigo[] enemigos;
    private final int largo;
    private int indice;
    /**
     * Crea un iterador de enemigos.
     * @param enemigos el array a recorrer
     * @param largo la cantidad de elementos del array a recorrer
     */
    public IteradorEnemigos(Enemigo[] enemigos, int largo) {
        this.enemigos = new Enemigo[largo];
        System.arraycopy(enemigos, 0, this.enemigos, 0, largo);
        this.largo = largo;
    }
    /**
     * Indica si aún tiene otro enemigo presente para devolver.
     * @return true si hay otro enemigo para devolver
     */
    public boolean tieneOtro() { return indice < largo; }
    /**
     * Devuelve el proximo enemigo.
     * @return el proximo enemigo
     */
}

```



```

    */
    public Enemigo proximo(){ return enemigos[indice++]; }
}

```

Clase Fondo

```

package juego;
import entorno.Entorno;
import java.awt.*;
//import java.time.Instant;
/**
 * Clase creada para el fondo del juego.
 *
 * @author Noelia Barrionuevo
 */
public class Fondo {
    private final double x;
    private final double y;
    private final Image fondo;
    /**
     * Crea un nuevo fondo.
     * @param x la coordenada x
     * @param y la coordenada y
     * @param fondo la imagen del fondo
     */
    public Fondo(double x, double y, Image fondo) {
        this.x = x;
        this.y = y;
        this.fondo = fondo;
    }
    /**
     * Dibuja el fondo.
     * @param entorno el entorno
     */
    public void dibujar(Entorno entorno) { entorno.dibujarImagen(fondo, x, y, 0); }
}

```

Clase Isla

```

package juego;
import entorno.Entorno;
import java.awt.*;
/**
 * Representa una isla flotante del juego.
 *
 * @author Noelia Barrionuevo
 * @author Miguel Angel Luna Lobos
 */
public class Isla {
    private final double x;
    private final double y;
    private final double ancho;
    private final double alto;
}

```

```

private final Image isla;
public Isla(double x, double y, double ancho, double alto, Image isla) {
    this.x = x;
    this.y = y;
    this.ancho = ancho;
    this.alto = alto;
    this.isla = isla;
}
/**
 * La coordenada x.
 * @return la coordenada x
 */
public double x() { return x; }
/**
 * La coordenada y.
 * @return la coordenada y
 */
public double y() { return y; }
/**
 * Dibuja la isla.
 * @param entorno el entorno
 * @param mundo el mundo
 */
public void dibujar(Entorno entorno, Mundo mundo) {
    final var coordenadasRelativas = Coordenadas.transformar(this.x, this.y, mundo,
entorno);
    final var x = coordenadasRelativas.x();
    final var y = coordenadasRelativas.y();
    entorno.dibujarImagen(isla, x, y, 0);
}
/**
 * Actúa sobre la princesa.
 * @param princesa la princesa
 */
public void actuarSobrePrincesa(Princesa princesa) {
    String tipoDeColision = Rectangulos.tipoDeColision(princesa.rectangulo(),
this.rectangulo());
    switch (tipoDeColision) {
        case "desde arriba":
            princesa.trasladar(princesa.x(), y() - alto / 2.0 - princesa.alto() /
2.0);
            princesa.recibirMensaje("estas en tierra firme");
            break;
        case "desde abajo":
            princesa.recibirMensaje("chocaste con el techo");
            break;
        case "desde la derecha":
            princesa.recibirMensaje("chocaste con un muro desde tu derecha");
            break;
        case "desde la izquierda":
            princesa.recibirMensaje("chocaste con un muro desde tu izquierda");
            break;
        default:
    }
}

```

```

        throw new IllegalArgumentException("tipo de colisión %s no
válido".formatted(tipoDeColision));
    }
}

/**
 * Actúa sobre un proyectil de la princesa.
 * @param proyectil el proyectil
 */
public void actuarSobreProyectilPrincesa(ProyectilPrincesa proyectil) {
    String tipoDeColision = Rectangulos.tipoDeColision(proyectil.rectangulo(),
this.rectangulo());
    switch (tipoDeColision) {
        case "desde arriba":
            proyectil.recibirMensaje("rebotar desde arriba");
            break;
        case "desde abajo":
            proyectil.recibirMensaje("rebotar desde abajo");
            break;
        case "desde la derecha":
            proyectil.recibirMensaje("rebotar desde la derecha");
            break;
        case "desde la izquierda":
            proyectil.recibirMensaje("rebotar desde la izquierda");
            break;
        default:
            throw new IllegalArgumentException("tipo de colisión %s no
soportado".formatted(tipoDeColision));
    }
}

/**
 * El rectángulo de colisión de la isla.
 * @return el rectángulo de colisión de la isla
 */
public Rectangulo rectangulo() { return new Rectangulo(x,y,ancho,alto); }
}

```

Clase Islas

```

package juego;
import java.awt.*;

/**
 * La clase que se encarga de generar islas en el mapa de manera aleatoria.
 *
 * @author Noelia Barrionuevo
 * @author Miguel Angel Luna Lobos
 */
public class Islas {
    public static final double proporcionAlto = 0.8;
    public static final double proporcionAlturaMinima = 0.0;
    public static final double altoIsla = 20.0;
    public static final double anchoMinimo = 400.0;
    public static final double anchoMaximo = 500.0;
    public static final double factorFronteraAncho = 2.0;
}

```

```

public static final double factorFronteraAlto = 1.0;
public static final int niveles = 8;
public static final double proporcionNivelesAltos = 0.6;
private static final double anchoIslaNivelBajo = 200.0;
public static final double proporcionIslasBajas = 10.0 / (800.0 * 600.0);
public static final double proporcionIslasAltas = 10.0 / (800.0 * 600.0);

private static Image crearImagenIsla(double ancho){
    return Imagenes.cargarYEscalar("isla.png", ancho, altoIsla);
}

public static Isla nuevaNivelBajo(Mundo mundo) {
    var isla = islaAleatoriaNivelBajo(mundo);
    var intentos = 0;
    while (mundo.islasEnColision(isla.rectangulo(), "fronteraIsla").length > 0) {
        isla = islaAleatoriaNivelBajo(mundo);
        if (intentos++ == 1000) {
            System.out.println("advertencia, no se pudo generar una isla de nivel
bajo");
            return null;
        }
    }
    return isla;
}

public static Isla nuevaNivelAlto(Mundo mundo) {
    var isla = islaAleatoriaNivelAlto(mundo);
    var intentos = 0;
    while (mundo.islasEnColision(isla.rectangulo(), "fronteraIsla").length > 0) {
        isla = islaAleatoriaNivelAlto(mundo);
        if (intentos++ == 1000) {
            System.out.println("advertencia, no se pudo generar una isla de nivel
alto");
            return null;
        }
    }
    return isla;
}

private static Isla islaAleatoriaNivelBajo(Mundo mundo) {
    final var anchoMundo = mundo.limiteMundo().ancho();
    final var altoMundo = mundo.limiteMundo().alto();
    final var alto = proporcionAlto * altoMundo;
    final var alturaMinima = proporcionAlturaMinima * altoMundo;
    final var yMax = altoMundo - alturaMinima;
    final var yMin = altoMundo - alturaMinima - alto;
    final var indice = Aleatorio.enteroRandom((int) Math.floor(niveles *
proporcionNivelesAltos), niveles);
    final var rango = yMax - yMin;
    final var y = indice * rango / (niveles - 1) + yMin;
    final var ancho = anchoIslaNivelBajo;
    final var xMin = 0.0 + ancho / 2.0;
    final var xMax = anchoMundo - ancho / 2.0;
}

```

```

        final var x = Aleatorio.decimalRandom(xMin, xMax);
        return new Isla(
            x,
            y,
            anchoIslaNivelBajo,
            altoIsla,
            crearImagenIsla(anchoIslaNivelBajo)
        );
    }

    private static Isla islaAleatoriaNivelAlto( Mundo mundo) {
        final var anchoMundo = mundo.limiteMundo().ancho();
        final var altoMundo = mundo.limiteMundo().alto();
        final var alto = proporcionAlto * altoMundo;
        final var alturaMinima = proporcionAlturaMinima * altoMundo;
        final var yMax = altoMundo - alturaMinima;
        final var yMin = altoMundo - alturaMinima - alto;
        final var indice = Aleatorio.enteroRandom(0, (int) Math.floor(niveles *
proporcionNivelesAltos));
        final var rango = yMax - yMin;
        final var y = indice * rango / (niveles - 1) + yMin;
        final var ancho = Aleatorio.decimalRandom(anchoMinimo, anchoMaximo);
        final var xMin = 0.0 + ancho / 2.0;
        final var xMax = anchoMundo - ancho / 2.0;
        final var x = Aleatorio.decimalRandom(xMin, xMax);
        return new Isla(x, y, ancho, altoIsla, crearImagenIsla(ancho));
    }
}

```

Clase Castillo

```

package juego;
import entorno.Entorno;
import java.awt.*;
/**
 * Clase del castillo.
 *
 * @author Miguel Angel Luna Lobos
 */
public class Castillo {
    private double x;
    private double y;
    private final double ancho;
    private final double alto;
    private final Image castillo;
    /**
     * Crea un nuevo castillo.
     * @param x la coordenada x
     * @param y la coordenada y
     * @param ancho el ancho
     * @param alto el alto
     * @param castillo la imagen del castillo
     */
}

```

```

    */
    public Castillo(double x, double y, double ancho, double alto, Image castillo){
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.castillo = castillo;
    }
    /**
     * Dibuja el castillo.
     * @param entorno el entorno
     * @param mundo el mundo
     */
    public void dibujar(Entorno entorno, Mundo mundo){
        final var coordenadasRelativas = Coordenadas.transformar(this.x, this.y, mundo,
entorno);
        final var x = coordenadasRelativas.x();
        final var y = coordenadasRelativas.y();
        entorno.dibujarImagen(castillo, x, y, 0);
    }
    /**
     * Traslada el castillo a nuevo punto x, y.
     * @param x la coordenada x
     * @param y la coordenada y
     */
    public void trasladar(double x, double y){
        this.x = x;
        this.y = y;
    }
    public double alto(){
        return alto;
    }
    public Rectangulo rectangulo(){
        return new Rectangulo(x, y, ancho * 0.2, alto);
    }
}

```

Clase Enemigo

```

package juego;
import entorno.Entorno;
import java.awt.*;
import java.time.Instant;
/**
 * Representa a un enemigo del juego.
 *
 * @author Noelia Barrionuevo
 * @author Miguel Angel Luna Lobos
 */
public class Enemigo {
    private double x;
    private final double y;
    private final double ancho;

```

```

private final double alto;
private final Image enemigo;
private final int id;
private final double velocidad;
private final double angulo;
private boolean vivo = true;
/**
 * Crea un nuevo enemigo.
 * @param generadorId el generador de identificadores únicos
 * @param x la coordenada x
 * @param y la coordenada y
 * @param ancho el ancho
 * @param alto el alto
 * @param angulo el ángulo
 * @param enemigo la imagen del enemigo
 */
Enemigo(GeneradorId generadorId, double x, double y, double ancho, double alto,
double angulo, Image enemigo) {
    this.x = x;
    this.y = y;
    this.ancho = ancho;
    this.alto = alto;
    this.enemigo = enemigo;
    id = generadorId.nuevoId();
    velocidad = 1.0;
    this.angulo = angulo;
}
/**
 * El identificador del enemigo.
 * @return el identificador del enemigo
 */
public int id() {
    return id;
}
/**
 * Dibuja al enemigo.
 * @param entorno el entorno
 * @param mundo el mundo
 */
public void dibujar(Entorno entorno, Mundo mundo) {
    final var coordenadasRelativas = Coordenadas.transformar(this.x, this.y, mundo,
entorno);
    final var x = coordenadasRelativas.x();
    final var y = coordenadasRelativas.y();
    entorno.dibujarImagen(enemigo, x, y, 0);
}
/**
 * Mueve al enemigo.
 */
public void mover() {
    x = x + velocidad * Math.cos(angulo);
}
/**

```

```

    * Recibe un mensaje.
    * @param mensaje
    */
    public void recibirMensaje(String mensaje) {
        if (mensaje.equals("morir")) {
            vivo = false;
        }
    }
    /**
     * Indica si este enemigo debe eliminarse.
     * @return true si debe eliminarse, false de lo contrario
     */
    public boolean debeEliminarse() { return !vivo; }
    /**
     * El rectángulo de colisión del enemigo.
     * @return el rectángulo de colisión del enemigo
     */
    public Rectangulo rectangulo() {
        return new Rectangulo(x, y, ancho, alto);
    }
}

```

Clase Enemigos

```

package juego;
import entorno.Entorno;
import java.awt.*;
/**
 * Clase con métodos utilitarios relacionados con enemigos.
 *
 * @author Miguel Angel Luna Lobos
 * @author Noelia Barrionuevo
 */
public class Enemigos {
    private static final double altoEnemigo = 40.0;
    private static final double anchoEnemigo = 40.0;
    public static final int minimoEnemigos = Islas.niveles / 2;
    private static final Image imagenEnemigo = Imagenes.cargarYEscalar("enemigo.png",
anchoEnemigo, altoEnemigo);
    /**
     * Crea un nuevo enemigo que ingresa en pantalla por la derecha.
     * @param generadorId el generador de identificadores únicos
     * @param mundo el mundo
     * @param entorno el entorno
     * @return un nuevo enemigo por derecha
     */
    public static Enemigo nuevoEnemigoDerecha(GeneradorId generadorId, Mundo mundo,
Entorno entorno) {
        var princesa = mundo.princesa();
        var enemigo = enemigoAleatorio(generadorId, mundo, princesa.x() + entorno.ancho()
/ 2.0, Math.PI);
        var intentos = 0;
        while (mundo.enemigosEnColision(enemigo.rectangulo()).length > 0) {

```



```

        enemigo = enemigoAleatorio(generatorId, mundo, princesa.x() + entorno.ancho()
/ 2.0, Math.PI);
        if (intentos++ == 100) {
            System.out.println("advertencia, no se pudo generar un enemigo");
            return null;
        }
    }
    return enemigo;
}

/**
 * Crea un nuevo enemigo que ingresa en pantalla por la derecha.
 * @param generatorId el generador de identificadores únicos
 * @param mundo el mundo
 * @param entorno el entorno
 * @return un nuevo enemigo por izquierda
 */
public static Enemigo nuevoEnemigoIzquierda(GeneradorId generatorId, Mundo mundo,
Entorno entorno) {
    var princesa = mundo.princesa();
    var enemigo = enemigoAleatorio(generatorId, mundo, princesa.x() - entorno.ancho()
/ 2.0, 0.0);
    var intentos = 0;
    while (mundo.enemigosEnColision(enemigo.rectangulo()).length > 0) {
        enemigo = enemigoAleatorio(generatorId, mundo, princesa.x() - entorno.ancho()
/ 2.0, 0.0);
        if (intentos++ == 100) {
            System.out.println("advertencia, no se pudo generar un enemigo");
            return null;
        }
    }
    return enemigo;
}

private static Enemigo enemigoAleatorio(GeneradorId generatorId, Mundo mundo, double
x, double angulo) {
    final var altoMundo = mundo.limiteMundo().alto();
    final var alto = Islas.proporcionAlto * altoMundo;
    final var alturaMinima = Islas.proporcionAlturaMinima * altoMundo;
    final var yMax = altoMundo - alturaMinima;
    final var yMin = altoMundo - alturaMinima - alto;
    final var y = alturaAleatoria(Islas.niveles, yMin, yMax);

    return new Enemigo(generatorId, x, y, anchoEnemigo, altoEnemigo, angulo,
imagenEnemigo);
}

private static double alturaAleatoria(int n, double yMin, double yMax) {
    if (n < 2) {
        throw new IllegalArgumentException("n no puede ser inferior a 2");
    }
    var q = Aleatorio.enteroRandom(0, n - 1);
    var indice = 2 * q + 1;

```

```

        var rango = yMax - yMin;
        return indice * rango / (2 * (n - 1)) + yMin;
    }
}

```

Clase Princesa

```

package juego;
import entorno.Entorno;
import java.awt.*;
import java.time.Instant;
import entorno.Herramientas;
/**
 * Personaje principal del juego.
 *
 * @author Miguel Angel Luna Lobos
 */
public class Princesa {
    public static final double altoPrincesa = 100;
    public static final double anchoPrincesa = 100;
    public static final double ladoCorazon = 30.0;
    private final Image princesa;
    private final Image corazon;
    private double x;
    private double y;
    private final double ancho;
    private final double alto;
    private double angulo;
    private final double velocidad;
    private double velocidadCaidaLibre;
    private final double velocidadSalto;
    private boolean enSalto;
    private double aceleracionGravitatoria;
    private Instant marcaTemporalDeCaida;
    private boolean xNoCrece;
    private boolean xNoDecrece;
    private int vidas;

    /**
     * Crea a la princesa en el centro horizontal de la pantalla e inicia la simulación
     * de caída libre.
     *
     * @param x la coordenada x
     * @param y la coordenada y
     * @param ancho el ancho
     * @param alto el alto
     * @param princesa la imagen de la princesa
     * @param corazon la imagen del corazón
     */
    public Princesa(double x, double y, double ancho, double alto, Image princesa, Image
corazon) {
        this.x = x;
        this.y = y;
    }
}

```

```

        this.alto = alto;
        this.ancho = ancho;
        this.princesa = princesa;
        this.corazon = corazon;
        vidas = 10;
        xNoCrece = false; // booleano que indica que no se puede avanzar hacia la derecha
        xNoDecrece = false; // booleano que indica que no se puede avanzar hacia la
izquierda
        velocidad = 3.0; // la velocidad de movimientos laterales
        velocidadSalto = 7.5; // la velocidad que alcanza la princesa tras saltar
        enSalto = false; // indica si la princesa está en un salto
        angulo = 0.0; // el ángulo en el que se mueve la princesa
        aceleracionGravitatoria = 10.0; // la constante g de este mundo
        velocidadCaídaLibre = 0.0; // la velocidad en caída libre, comienza en cero
        marcaTemporalDeCaída = Instant.now(); // iniciamos con la princesa en caída libre
    }
    public int vidas() { return vidas; }
    public void trasladar(double x, double y) {
        this.x = x;
        this.y = y;
        marcaTemporalDeCaída = Instant.now();
    }
    /**
     * La coordenada x.
     * @return la coordenada x
     */
    public double x() { return x; }
    /**
     * La coordenada y.
     * @return la coordenada y
     */
    public double y() { return y + alto * 0.05; }
    /**
     * El ancho.
     * @return el ancho
     */
    public double ancho() { return ancho * 0.45; }
    /**
     * El alto.
     * @return el alto
     */
    public double alto() { return alto * 0.80; }
    public void dibujar(Entorno entorno) {
        var espacio = 5.0;
        for (var i = 0; i < vidas; i++) {
            double l = ladoCorazon * (i + 0.5) + (i + 1) * 5.0;
            entorno.dibujarImagen(
                corazon,
                l,
                espacio + ladoCorazon / 2.0,
                0,
                1
            );
        }
    }

```

```

    }
    final var dx = entorno.ancho() / 2.0;
    final var dy = entorno.alto() / 2.0;
    entorno.dibujarImagen(princesa, dx, dy, 0, 1);
}
/**
 * Ejecuta el movimiento de la princesa.
 * @param entorno el entorno
 */
public void mover(Entorno entorno) {
    if (entorno.estaPresionada(entorno.TECLA_DERECHA)) {
        angulo = 0;
        movimientoLateral();
    }
    if (entorno.estaPresionada(entorno.TECLA_IZQUIERDA)) {
        angulo = Math.PI;
        movimientoLateral();
    }
    if ((entorno.sePresiono('a') || entorno.estaPresionada(entorno.TECLA_ARRIBA)) &&
    aceleracionGravitatoria <= 0.1) {
        enSalto = true;
        cayendo();
    }
    gravedad();
    if (enSalto) {
        movimiento(-Math.PI / 2, velocidadSalto);
    }
}
/**
 * Aplica el movimiento lateral de la princesa, reduciendo la velocidad si está
 * en tierra firme para evitar deslizamiento excesivo.
 */
private void movimientoLateral() {
    if (aceleracionGravitatoria <= 0.1) {
        movimiento(angulo, velocidad * 0.75);
    } else {
        movimiento(angulo, velocidad);
    }
}
/**
 * Desplaza a la princesa en la dirección y velocidad indicadas, sin ninguna
 * condición adicional.
 *
 * @param angulo    el ángulo de desplazamiento en radianes
 * @param velocidad la velocidad de desplazamiento en píxeles por frame
 */
private void movimiento(double angulo, double velocidad) {
    double deltaX = Math.cos(angulo) * velocidad;

    if (deltaX > 0.0 && xNoCrece) {
        deltaX = 0.0;
        xNoCrece = false;
    } else if (deltaX < 0 && xNoDecrece) {

```

```

        deltaX = 0.0;
        xNoDecrece = false;
    }

    x += deltaX;
    y += Math.sin(angulo) * velocidad;
}
/**
 * Aplica la simulación de gravedad: acumula velocidad de caída libre con el tiempo
 * y la anula cuando la princesa está en tierra firme.
 *
 */
private void gravedad() {
    if (aceleracionGravitatoria <= 0.1) {
        marcaTemporalDeCaida = Instant.now();
    } else {
        double lapso = Instant.now().toEpochMilli() -
marcaTemporalDeCaida.toEpochMilli();
        velocidadCaidaLibre = aceleracionGravitatoria * lapso / 1000;
        movimiento(Math.PI / 2, velocidadCaidaLibre);
    }
    aceleracionGravitatoria = 10.0;
}

/**
 * Recibe mensajes.
 * @param mensaje el mensaje
 */
public void recibirMensaje(String mensaje) {

    switch (mensaje) {
        case "una vida menos": // :C
            vidas--;
            break;
        case "estas en tierra firme":
            enTierraFirme();
            break;
        case "chocaste con el techo":
            chocarTecho();
            break;
        case "chocaste con un muro desde tu derecha":
            chocarMuroPorDerecha();
            break;
        case "chocaste con un muro desde tu izquierda":
            chocarMuroPorIzquierda();
            break;
        case "morir":
            vidas = 0;
            break;
        default:
            throw new IllegalArgumentException("así no se le habla a la princesa ->
mensaje: " + mensaje);
    }
}

```

```

    }
    private void chocarMuroPorIzquierda() {
        enSalto = false;
        xNoDecrece = true;
    }
    private void chocarTecho() {
        enSalto = false;
    }
    private void chocarMuroPorDerecha() {
        enSalto = false;
        xNoCrece = true;
    }
    private void cayendo() {
        marcaTemporalDeCaida = Instant.now();
        velocidadCaidaLibre = 0.0;
    }
    private void enTierraFirme() {
        aceleracionGravitatoria = 0.0;
        enSalto = false;
        marcaTemporalDeCaida = null;
        velocidadCaidaLibre = 0.0;
    }
    /**
     * Dispara un proyectil.
     * @param mundo el mundo
     * @param entorno el entorno
     */
    public void disparar(Mundo mundo, Entorno entorno) {
        final var mouseX = entorno.mouseX() + x - entorno.ancha() / 2.0;
        final var mouseY = entorno.mouseY() + y - entorno.alto() / 2.0;
        final var distanciaX = mouseX - x;
        final var distanciaY = mouseY - y;
        final var distancia = Math.sqrt(Math.pow(distanciaX, 2.0) + Math.pow(distanciaY,
2.0));
        final var cos = (distanciaX) / distancia;
        final var sin = (distanciaY) / distancia;
        final var proyectil = new ProyectilPrincesa(x, y, cos, sin);
        // se agrega sonido cuando la princesa dispara
        // se llamo a la clase de herramientas play
        try {
            Herramientas.play("sonido/sonidoPoder.wav");
        }
        catch (Exception error){
            System.out.println ("No se puede reproducir sonido");
        }
        mundo.establecerProyectilPrincesa(proyectil);
    }

    /**
     * El rectángulo de colisión
     * @return el rectángulo de colisión
     */
    public Rectangulo rectangulo() {

```

```

        return new Rectangulo(x(), y(), ancho(), alto());
    }
}

```

Clase Jefe

```

package juego;
import entorno.Entorno;
import java.awt.*;
/**
 * Representa al jefe.
 *
 * @author Noelia Barrionuevo
 * @author Miguel Angel Luna Lobos
 */
public class Jefe {
    public static final double altoJefe = 130;
    public static final double anchoJefe = 120;
    private double x;
    private double y;
    private final double ancho;
    private final double alto;
    private final Image jefeHaciaDerecha;
    private final Image jefeHaciaIzquierda;
    private Image jefe;
    private double cos;
    private final double velocidad;
    private int vidas;
    private Isla isla;
    public Jefe(double x, double y, double ancho, double alto, Image jefeHaciaDerecha,
Image jefeHaciaIzquierda) {
        this.x = x;
        this.y = y;
        this.ancho = ancho;
        this.alto = alto;
        this.jefeHaciaDerecha = jefeHaciaDerecha;
        this.jefeHaciaIzquierda = jefeHaciaIzquierda;
        this.jefe = jefeHaciaIzquierda;
        velocidad = 1.0;
        cos = Math.cos(Math.PI);
        isla = null;
        vidas = 10;
    }

    public void establecerIsla(Isla isla){
        this.isla = isla;
        this.x = isla.x();
        this.y = isla.y() - alto / 2.0 - Islas.altoIsla / 2.0;
    }

    public double x() { return x; }
    public double y() { return y; }
    public double ancho() { return ancho; }
}

```

```

public double alto() { return alto; }
public int vidas(){ return vidas; }
/**
 * Dibuja al jefe.
 *
 * @param entorno el entorno
 * @param mundo   el mundo
 */
public void dibujar(Entorno entorno, Mundo mundo) {
    final var coordenadasRelativas = Coordenadas.transformar(this.x, this.y, mundo,
entorno);
    final var x = coordenadasRelativas.x();
    final var y = coordenadasRelativas.y();
    final var rectanguloRojo = new Rectangulo(x, y - alto / 2.0, 100, 10);
    double proporcion = vidas / 10.0;
    double diferencia = rectanguloRojo.anch() - proporcion * 100;
    final var rectanguloNegro = new Rectangulo(x - diferencia / 2, y - alto / 2.0,
100 * proporcion, 10);
    rectanguloRojo.dibujarRectangulo(entorno, Color.RED);
    rectanguloNegro.dibujarRectangulo(entorno, Color.BLACK);
    entorno.dibujarImagen(jefe, x, y, 0);
}
public void mover() {
    if(x <= isla.rectangulo().bordeIzquierdo() || x >=
isla.rectangulo().bordeDerecho()){
        cos = -cos;
        if(cos < 0){
            jefe = jefeHaciaIzquierda;
        } else {
            jefe = jefeHaciaDerecha;
        }
    }
    x += velocidad * cos;
}
/**
 * Recibe mensajes.
 *
 * @param mensaje el mensaje
 */
public void recibirMensaje(String mensaje) {
    if (mensaje.equals("una vida menos")) {
        vidas--;
    }
}
/**
 * El rectángulo de colisión.
 * @return el rectángulo de colisión
 */
public Rectangulo rectangulo() {
    return new Rectangulo(x, y, ancho, alto);
}
}

```


Clase Mundo

```
package juego;
import java.util.Objects;
/**
 * Contiene los objetos que se van dibujando en pantalla.
 *
 * @author Miguel Angel Luna Lobos
 * @author Noelia Barrionuevo
 */
public class Mundo {
    public static final double proporcionAnchoMundo = 8.0;
    public static final double proporcionAltoMundo = 2.0;
    private final Princesa princesa;
    private Enemigo[] enemigos;
    private Isla[] islas;
    private Jefe jefe;
    private ProyectoilPrincesa proyectilPrincesa;
    private final Castillo castillo;
    private int largoEnemigos;
    private int largoIslas;
    private final Rectangulo limitesMundo;
    private final Rectangulo limitesPantalla;
    private final Fondo fondo;

    /**
     * Crea una nueva instancia de Mundo.
     * @param limitesPantalla el rectángulo que indica los límites de la pantalla
     * @param limitesMundo el rectángulo que indica los límites del mundo
     * @param princesa la princesa
     * @param jefe el jefe
     * @param fondo el fondo
     * @param castillo el castillo
     */
    public Mundo(
        Rectangulo limitesPantalla,
        Rectangulo limitesMundo,
        Princesa princesa,
        Jefe jefe,
        Fondo fondo,
        Castillo castillo
    ) {

        this.limitesPantalla = Objects.requireNonNull(limitesPantalla, "limitesPantalla no puede ser null");
        this.limitesMundo = Objects.requireNonNull(limitesMundo, "limitesMundo no puede ser null");
        this.princesa = Objects.requireNonNull(princesa, "princesa no puede ser null");
        this.jefe = jefe;
        this.fondo = Objects.requireNonNull(fondo, "fondo no puede ser null");
        this.castillo = Objects.requireNonNull(castillo, "castillo no puede ser null");
        islas = new Isla[10];
        enemigos = new Enemigo[10];
    }
}
```

```

        largoEnemigos = 0;
        largoIslas = 0;
    }
    /**
     * Los límites del mundo.
     * @return un rectángulo que representa los límites del mundo
     */
    public Rectangulo limitesMundo(){ return limitesMundo; }
    /**
     * Los límites de la pantalla.
     * @return un rectángulo que representa los límites de la pantalla.
     */
    public Rectangulo limitesPantalla() {
        return new Rectangulo(princesa.x(), princesa.y(), limitesPantalla.ancho(),
limitesPantalla.alto());
    }
    /**
     * El fondo del juego.
     * @return el fondo
     */
    public Fondo fondo(){ return fondo; }
    /**
     * La princesa.
     * @return la princesa
     */
    public Princesa princesa() { return princesa; }
    /**
     * El castillo.
     * @return el castillo
     */
    public Castillo castillo() { return castillo; }
    /**
     * El jefe.
     * @return el jefe o null si no existe
     */
    public Jefe jefe() { return jefe; }
    /**
     * El proyectil que disparó la princesa. Puede ser null.
     * @return el proyectil disparado por la princesa, o null si no existe
     */
    public ProyectilPrincesa proyectilPrincesa(){ return proyectilPrincesa; }
    /**
     * Devuelve un array con las islas en colisión con el rectángulo provisto.
     * @param rectangulo el rectángulo de colisión
     * @param tipo el tipo de elemento
     * @return un array que lista las islas en colisión con el rectángulo argumento
     */
    public Isla[] islasEnColision(Rectangulo rectangulo, String tipo){
        if (tipo.equals("fronteraIsla")) {
            var frontera = rectangulo.escalar(Islas.factorFronteraAncho,
Islas.factorFronteraAlto);
            return enColisionIsla(frontera);
        }
    }

```

```

        final Isla[] almacenador = new Isla[largoIslas];
        int contador = 0;
        for (int i = 0; i < largoIslas; i++) {
            Isla isla_ = islas[i];
            if (Rectangulos.enColision(rectangulo, isla_.rectangulo())) {
                almacenador[contador++] = isla_;
            }
        }
        final Isla[] salida = new Isla[contador];
        System.arraycopy(almacenador, 0, salida, 0, contador);
        return salida;
    }

    private Isla[] enColisionIsla(Rectangulo rectangulo) {
        final Isla[] almacenador = new Isla[largoIslas];
        int contador = 0;
        for (int i = 0; i < largoIslas; i++) {
            var isla = islas[i];
            var frontera = isla.rectangulo().escalar(Islas.factorFronteraAncho,
Islas.factorFronteraAlto);
            if (Rectangulos.enColision(rectangulo, frontera)) {
                almacenador[contador++] = isla;
            }
        }
        final Isla[] salida = new Isla[contador];
        System.arraycopy(almacenador, 0, salida, 0, contador);
        return salida;
    }
}

/**
 * Devuelve un array con los enemigos en colisión con el rectángulo provisto.
 * @param rectangulo el rectángulo de colisión
 * @return un array que lista los enemigos en colisión con el rectángulo argumento
 */
public Enemigo[] enemigosEnColision(Rectangulo rectangulo){
    final Enemigo[] almacenador = new Enemigo[largoEnemigos];
    int contador = 0;
    for (int i = 0; i < largoEnemigos; i++) {
        Enemigo enemigo = enemigos[i];
        if (Rectangulos.enColision(rectangulo, enemigo.rectangulo())) {
            almacenador[contador++] = enemigo;
        }
    }
    final Enemigo[] salida = new Enemigo[contador];
    System.arraycopy(almacenador, 0, salida, 0, contador);
    return salida;
}

/**
 * Agrega un enemigo.
 * @param enemigo el enemigo a agregar
 */
public void agregarEnemigo(Enemigo enemigo) {
    if (largoEnemigos == enemigos.length) {
        Enemigo[] nuevoArray = new Enemigo[enemigos.length * 2];
        System.arraycopy(enemigos, 0, nuevoArray, 0, enemigos.length);
    }
}

```

```

        enemigos = nuevoArray;
    }
    enemigos[largoEnemigos++] = enemigo;
}

/**
 * Agrega una isla
 * @param isla la isla agregar
 */
public void agregarIsla(Isla isla) {
    if (largoIslas == islas.length) {
        Isla[] nuevoArray = new Isla[islal.length * 2];
        System.arraycopy(islas, 0, nuevoArray, 0, islas.length);
        islas = nuevoArray;
    }
    islas[largoIslas++] = isla;
}

/**
 * Quita un enemigo.
 * @param enemigo el enemigo a quitar
 */
public void quitarEnemigo(Enemigo enemigo) {
    int indice = indiceDeEnemigo(enemigo);
    if (indice >= 0) {
        for (int i = indice; i < largoEnemigos - 1; i++) {
            enemigos[i] = enemigos[i + 1];
        }
        enemigos[largoEnemigos - 1] = null;
        largoEnemigos--;
    }
}

private int indiceDeEnemigo(Enemigo enemigo) {
    int indiceMinimo = 0;
    int indiceMaximo = largoEnemigos - 1;
    while (indiceMinimo <= indiceMaximo) {
        int indiceIntermedio = (indiceMinimo + indiceMaximo) / 2;
        Enemigo elementoIntermedio = enemigos[indiceIntermedio];
        int comparacion = compararEnemigos(elementoIntermedio, enemigo);

        if (comparacion < 0) {
            indiceMinimo = indiceIntermedio + 1;
        } else if (comparacion > 0) {
            indiceMaximo = indiceIntermedio - 1;
        } else {
            return indiceIntermedio; // Encontrado (comparacion == 0)
        }
    }
    return -(indiceMinimo + 1);
}

private int compararEnemigos(Enemigo enemigo1, Enemigo enemigo2) {
    if (enemigo1.id() < enemigo2.id()) {
        return -1;
    } else if (enemigo1.id() == enemigo2.id()) {

```

```

        return 0; // esta condición nunca debería ocurrir porque los ids son únicos
    } else {
        return 1;
    }
}

/**
 * Se ocupa de eliminar a los enemigos que se salén de la pantalla así como de
eliminar a los que
 * deben morir por otras razones.
 */
public void purgar() {
    int i = 0;
    while (i < largoEnemigos) {
        var enemigo = enemigos[i];
        if (!Rectangulos.enColision(enemigo.rectangulo(), limitesPantalla()) ||
enemigo.debeEliminarse()) {
            quitarEnemigo(enemigo);
        } else {
            i++;
        }
    }
    if(jefe != null && jefe.vidas() <= 0){
        jefe = null;
    }
    if(proyectilPrincesa != null &&
!Rectangulos.enColision(proyectilPrincesa.rectangulo(), limitesPantalla())){
        proyectilPrincesa = null;
    }
}

/**
 * Establece el proyectil de la princesa.
 * @param proyectilPrincesa el proyectil de la princesa
 */
public void establecerProyectilPrincesa(ProyectilPrincesa proyectilPrincesa){
    this.proyectilPrincesa = proyectilPrincesa;
}

/**
 * Devuelve un iterador de islas.
 * @return un iterador que recorre todas las islas del mundo
 */
public IteradorIslas iteradorIslas(){
    return new IteradorIslas(islas, largoIslas);
}

/**
 * Devuelve un iterador de enemigos.
 * @return un iterador que recorre todos los enemigos del mundo
 */
public IteradorEnemigos iteradorEnemigos(){
    return new IteradorEnemigos(enemigos, largoEnemigos);
}

/**
 * Indica si faltan enemigos.

```

```

        * @return true si se deben generar más enemigos.
        */
    public boolean faltanEnemigos() {
        return largoEnemigos < Enemigos.minimoEnemigos;
    }
}

```

Clase Juego

```

package juego;
import entorno.Entorno;
import entorno.InterfaceJuego;
import java.awt.*;
import java.util.Objects;
import entorno.Herramientas ;
public class Juego extends InterfaceJuego {
    // El objeto Entorno que controla el tiempo y otros
    private final Entorno entorno;

    // Variables y métodos propios de cada grupo
    // ...
    private final GeneradorId generadorId;
    private final Mundo mundo;

    private boolean victoria;
    private boolean derrota;

    Juego() {
        // Inicializa el objeto entorno
        this.entorno = new Entorno(this, "Super Elizabeth Sis", 800, 600);

        // Inicializar lo que haga falta para el juego
        // ...

        generadorId = new GeneradorId();
        victoria = false;
        derrota = false;
        final var anchoPantalla = entorno.ancho();
        final var altoPantalla = entorno.alto();
        final var anchoMundo = anchoPantalla * Mundo.proporcionAnchoMundo;
        final var altoMundo = altoPantalla * Mundo.proporcionAltoMundo;
        final var limitesPantalla = new Rectangulo(anchoPantalla / 2.0, altoPantalla /
2.0, anchoPantalla, altoPantalla);
        final var limitesMundo = new Rectangulo(anchoMundo / 2.0, altoMundo / 2.0,
anchoMundo, altoMundo);
        final var imagenPrincesa = Imagenes.cargarYEscalar("princesa.png",
Princesa.anchoPrincesa, Princesa.altoPrincesa);
        final var imagenCorazon = Imagenes.cargarYEscalar("corazon.png",
Princesa.ladoCorazon, Princesa.ladoCorazon);
        final var princesa = new Princesa(0.0, 0.0, Princesa.anchoPrincesa,
Princesa.altoPrincesa, imagenPrincesa, imagenCorazon);
        final var imagenJefeHaciaDerecha =
Imagenes.cargarYEscalar("jefe_hacia_derecha.png", Jefe.anchoJefe, Jefe.altoJefe);

```

```

        final var imagenJefeHaciaIzquierda =
Imagenes.cargarYEscalar("jefe_hacia_izquierda.png", Jefe.anchoSJefe, Jefe.altoJefe);
        final var jefe = new Jefe( 0, 0, Jefe.anchoSJefe, Jefe.altoJefe,
imagenJefeHaciaDerecha, imagenJefeHaciaIzquierda);
        final var imagenFondo = Imagenes.cargarYEscalar("fondo.png", anchoPantalla,
altoPantalla);
        final var fondo = new Fondo(anchoPantalla / 2.0, altoPantalla / 2.0,
imagenFondo);
        final var imagenCastillo = Imagenes.cargarYEscalar("castillo.png",
Islas.anchoSMinimo, Islas.anchoSMinimo);
        final var castillo = new Castillo(0.0, 0.0, Islas.anchoSMinimo, Islas.anchoSMinimo,
imagenCastillo);
        mundo = new Mundo(lmitesPantalla, lmitesMundo, princesa, jefe, fondo,
castillo);

        var cantidadIslasBajas = Islas.proporcionIslasBajas *
mundo.lmitesMundo().area();
        var cantidadIslasAltas = Islas.proporcionIslasAltas *
mundo.lmitesMundo().area();
        var contador = 0;
        while (contador < cantidadIslasBajas) {
            var isla = Islas.nuevaNivelBajo(mundo);
            if (isla == null) {
                break;
            }
            mundo.agregarIsla(isla);
            contador++;
        }
        contador = 0;
        var xMin = mundo.lmitesMundo().ancho();
        var xMax = 0.0;
        var yMin = mundo.lmitesMundo().alto();
        Isla primeraIsla = null;
        Isla anteUltimaIsla = null;
        Isla ultimaIsla = null;
        while (contador < cantidadIslasAltas) {
            var isla = Islas.nuevaNivelAlto(mundo);

            if (isla == null) {
                break;
            }
            if (isla.x() < xMin) {
                xMin = isla.x();
                primeraIsla = isla;
            }
            if (isla.x() > xMax && isla.y() <= yMin) {
                xMax = isla.x();
                yMin = isla.y();
                anteUltimaIsla = ultimaIsla;
                ultimaIsla = isla;
            }

            mundo.agregarIsla(isla);

```

```

        contador++;
    }
    Objects.requireNonNull(primeraisla, "no se ha encontrado una isla en la cual
colocar a la princesa");
    Objects.requireNonNull(anteultimaIsla, "no se ha encontrado una isla en la cual
colocar al jefe");
    Objects.requireNonNull(ultimaIsla, "no se ha encontrado una isla en la cual
colocar el castillo");
    mundo.princesa().trasladar(primeraisla.x(), primeraisla.y() - Islas.altoIsla /
2.0 - mundo.princesa().alto() / 2.0);
    mundo.jefe().establecerIsla(anteultimaIsla);
    mundo.castillo().trasladar(ultimaIsla.x(), ultimaIsla.y() - Islas.altoIsla / 2.0
- mundo.castillo().alto() / 2.0);
    // Inicia el juego!
    this.entorno.iniciar();
}

/**
 * Durante el juego, el método tick() será ejecutado en cada instante y
 * por lo tanto es el método más importante de esta clase. Aquí se debe
 * actualizar el estado interno del juego para simular el paso del tiempo
 * (ver el enunciado del TP para mayor detalle).
 */
public void tick() {
    // Procesamiento de un instante de tiempo
    // ...
    mundo.fondo().dibujar(entorno);
    if (victoria) {
        dibujarEnemigos();
        colisionesIslas();
        colisionesJefe();
        colisionesProyectilPrincesa();
        colisionesCastillo();
        var x = mundo.princesa().x();
        var y = mundo.princesa().y();
        mundo.princesa().trasladar(x, y);
        entorno.cambiarFont("Arial", 72, Color.BLUE);
        entorno.escribirTexto("¡Ganaste!", entorno.ancho() / 2.0 - 160,
entorno.alto() / 2.0);
        return;
    }
    if (derrota) {
        dibujarEnemigos();
        colisionesIslas();
        colisionesJefe();
        colisionesProyectilPrincesa();
        colisionesCastillo();
        var x = mundo.princesa().x();
        var y = mundo.princesa().y();
        mundo.princesa().trasladar(x, y);
        entorno.cambiarFont("Arial", 72, Color.RED);
        entorno.escribirTexto("Perdiste", entorno.ancho() / 2.0 - 150, entorno.alto()
/ 2.0);
    }
}

```



```

        return;
    }
    if (mundo.faltanEnemigos()) {
        var opcion = Aleatorio.enteroRandom(0, 2);
        Enemigo enemigo;
        switch (opcion) {
            case 0:
                enemigo = Enemigos.nuevoEnemigoDerecha(generatorId, mundo, entorno);
                break;
            case 1:
                enemigo = Enemigos.nuevoEnemigoIzquierda(generatorId, mundo,
entorno);
                break;
            default:
                throw new RuntimeException("error desconocido");
        }

        if (enemigo != null) {
            mundo.agregarEnemigo(enemigo);
        }
    }
    mundo.purgar(); // eliminamos a aquellos que se salen del mundo
    if (entorno.sePresionoBoton(entorno.BOTON_IZQUIERDO) && mundo.proyectilPrincesa()
== null) {
        mundo.princesa().disparar(mundo, entorno);
    }
    colisionesIslas();
    colisionesJefe();
    colisionesProyectilPrincesa();
    dibujarEnemigos();
    colisionesCastillo();
    colisionesPrincesa();
}

private void dibujarEnemigos() {
    var iterador = mundo.iteradorEnemigos();
    while (iterador.tieneOtro()) {
        var enemigo = iterador.proximo();
        enemigo.mover();
        enemigo.dibujar(entorno, mundo);
    }
}

private void colisionesIslas() {
    var iterador = mundo.iteradorIslas();
    var princesa = mundo.princesa();
    var proyectil = mundo.proyectilPrincesa();
    while (iterador.tieneOtro()) {
        var isla = iterador.proximo();
        if (Rectangulos.enColision(isla.rectangulo(), princesa.rectangulo())) {
            isla.actuarSobrePrincesa(princesa);
        }
    }
}

```

```

        if (proyectil != null && Rectangulos.enColision(isla.rectangulo(),
proyectil.rectangulo())) {
            isla.actuarSobreProyectilPrincesa(proyectil);
        }
        isla.dibujar(entorno, mundo);
    }
}

private void colisionesJefe() {
    var jefe = mundo.jefe();
    if (jefe == null) {
        return;
    }

    var princesa = mundo.princesa();
    if (Rectangulos.enColision(jefe.rectangulo(), princesa.rectangulo())) {
        princesa.recibirMensaje("morir");
    }
    jefe.mover();
    jefe.dibujar(entorno, mundo);
}

private void colisionesProyectilPrincesa() {
    var proyectil = mundo.proyectilPrincesa();
    if (proyectil == null) {
        return;
    }
    var jefe = mundo.jefe();
    if (jefe != null && Rectangulos.enColision(jefe.rectangulo(),
proyectil.rectangulo())) {
        mundo.establecerProyectilPrincesa(null);
        jefe.recibirMensaje("una vida menos");
        // El dragon emite sonido cuando un
        //proyectil lo colisiona
        try {
            Herramientas.play("sonido/dragon.wav");
        }
        catch (Exception error){
            System.out.println ("No se puede reproducir sonido de dragon");
        }
    }
    proyectil = mundo.proyectilPrincesa();
    if(proyectil == null){
        return;
    }
    var enemigosEnColision = mundo.enemigosEnColision(proyectil.rectangulo());
    if(enemigosEnColision.length > 0){
        mundo.establecerProyectilPrincesa(null);
    }
    for (var i = 0; i < enemigosEnColision.length; i++) {
        enemigosEnColision[i].recibirMensaje("morir");
    }
    proyectil = mundo.proyectilPrincesa();
}

```

```

        if(proyectil == null){
            return;
        }
        proyectil.mover();
        proyectil.dibujar(entorno, mundo);
    }

private void colisionesPrincesa() {
    var princesa = mundo.princesa();
    if (princesa.vidas() <= 0) {
        derrota = true;
    }
    if (princesa.y() > mundo.limiteMundo().alto()*2.5) {
        //se agrega sonido cuando la princesa cae al vacio y pierde una vida
        try{
            Herramientas.play ("sonido/dragon2.wav");
        }catch (Exception error){
            System.out.println("Princesa cae al vacio no funciona el sonido");
        }
        princesaCaeAlVacio();
    }
    var enemigosEnColision = mundo.enemigosEnColision(princesa.rectangulo());
    for (int i = 0; i < enemigosEnColision.length; i++) {
        enemigosEnColision[i].recibirMensaje("morir");
        //la princesa emite sonido cuando le sacan vidas
        try{
            Herramientas.play ("sonido/PrincesaPierdeVidas.wav" );
        } catch (Exception error){
            System.out.println ("Sonido de pierde vidas con enemigo");
        }
        princesa.recibirMensaje("una vida menos");
    }
    princesa.mover(entorno);
    princesa.dibujar(entorno);
}

private void princesaCaeAlVacio(){
    var princesa = mundo.princesa();
    princesa.recibirMensaje("una vida menos");
    if(princesa.vidas() <= 0){
        derrota = true;
        return;
    }
    var iteradorIslas = mundo.iteradorIslas();
    var islaMasCercana = iteradorIslas.proximo();
    var deltaX = Math.abs(islaMasCercana.x() - princesa.x());
    while(iteradorIslas.tieneOtro()){
        var isla = iteradorIslas.proximo();
        var nuevoDeltaX = Math.abs(isla.x() - princesa.x());
        if(nuevoDeltaX <= deltaX){
            deltaX = nuevoDeltaX;
            islaMasCercana = isla;
        }
    }
}

```

```

        }
        princesa.trasladar(islaMasCercana.x(), islaMasCercana.y() - Islas.altoIsla / 2.0
-    princesa.alto() / 2.0);
    }

    private void colisionesCastillo() {
        var princesa = mundo.princesa();
        var castillo = mundo.castillo();
        if (Rectangulos.enColision(princesa.rectangulo(), castillo.rectangulo())) {
            victoria = true;
        }
        mundo.castillo().dibujar(entorno, mundo);
    }

    @SuppressWarnings("unused")
    public static void main(String[] args) {
        Juego juego = new Juego();
    }
}

```